

# STT810 Final Project- Bank Fraud

Dennis Ousmanov, Yun Chen, Ran Zheng

December 2025

## Abstract

Fraud Banking Transactions happen more often than we think in our day to day world. We are keen to find if there are ways for us to build methods to detect fraud. Therefore, we used a database from Kaggle and performed analysis on it. First, we did Exploratory Data Analysis (EDA) and found that only 1% of all the data is fraudulent. We then performed correlation analysis on the numerical data and created visualizations for the proportion of fraud/non-fraud transactions for different levels of the categorical columns. Later, we developed two models for predicting fraud. One model uses the XGBoost method, and another model uses the Random Forest method. Both models performed fairly, with XGBoost having a recall score of 0.61 for fraudulent cases and an overall f1-score of 0.70, and Random Forest recall score of 0.46 and an overall f1-score of 0.61.

## 1 Introduction

Fraudulent behaviors occur frequently in our lives, resulting in substantial loss for individuals and companies alike. An estimate mentioned that in the US alone, nearly 73% of adults encounter scams of some kind almost on a weekly basis (Jeffery, Eugenie and Monica, 2025).

While it is important to remain alert to fraudulent emails and online messages, some sources say that fraud is preventable when certain characteristics are identified (Editorial Journalist, 2025). It is demonstrated by researchers that through machine learning models such as Random Forest or Neural Networks, one may be able to predict whether a call or an email is a fraud or not.

Such an analysis greatly motivated us, as we are passionate about applying machine learning methods to real life problems. We are convinced that with the right approach, one can detect fraud before it even reaches them, so as to minimize the loss that might incur.

For this project, our team aims to explore the patterns of fraud, examine the correlations between different features and fraud, and determine whether we can generate a model to predict fraud given the datasets we have. In addition, we will use the ROC Curve to evaluate how well our models perform in terms of detecting fraud.

## 2 Dataset

We obtained our dataset on Kaggle, titled “Bank Account Fraud Dataset Suite”(Sérgio, Pedro, and Jose, 2022). These datasets, although synthetic, are designed to be robust and realistic for training machine learning models to predict fraud in real-world applications. A notable characteristic of this dataset is the strong class imbalance, in this case there are many more non-fraud than fraud cases, mirroring a realistic scenario.

The suite contains 6 datasets - one labeled **Base** and five variants (**Variant I** through **Variant V**). For convenience, we focus solely on the **Base** dataset for this research. The dataset has 1 million rows and 32 columns. The key indicator is the `fraud_bool` column, which is coded as 0 for non-fraud and 1 for fraud. We would like to examine which features exhibit a strong correlation with this indicator.

Some features in this dataset includes: income, proposed credit limit, device os, session length in minutes, and so on.

### 3 Exploratory Data Analysis

We found that out of all the 1 millions entries, only about 10 thousands of them are fraud, which is about 1% of all data. The proportion is shown in the Figure 1 below.

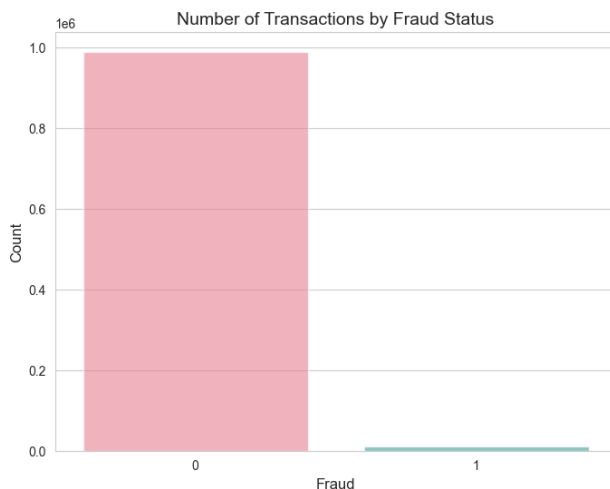


Figure 1: Number of Transactions by Fraud Status

Given this imbalance, our goal is to identify which features are most associated with fraudulent cases. To do so, we need to analyze both numerical columns and categorical columns.

#### 3.1 Missingness

Although no missing values were detected when we checked the dataset directly, the author of the documentation indicates that any value equal to -1 should be interpreted as a missing value. Given this finding, we encoded row, column values that are -1 as NaN. The proportion of values missing per column strata by our target class is shown below:

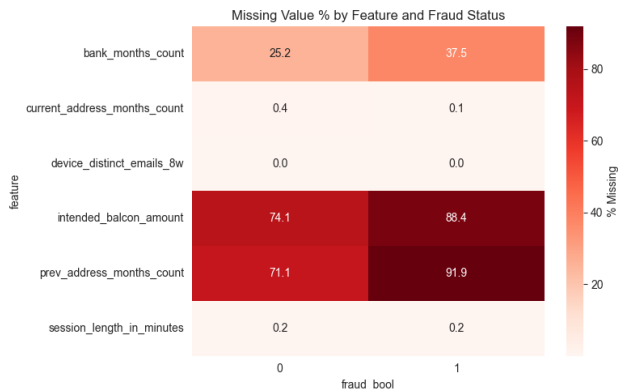


Figure 2: Missing Value

#### 3.2 Numerical columns

We are interested in finding the factors that correlate most strongly with fraud. First, we examine the numerical data and compute a correlation heatmap and eventually we found out most of the variables have a weak correlation to our target column. Figure 3 shows this.

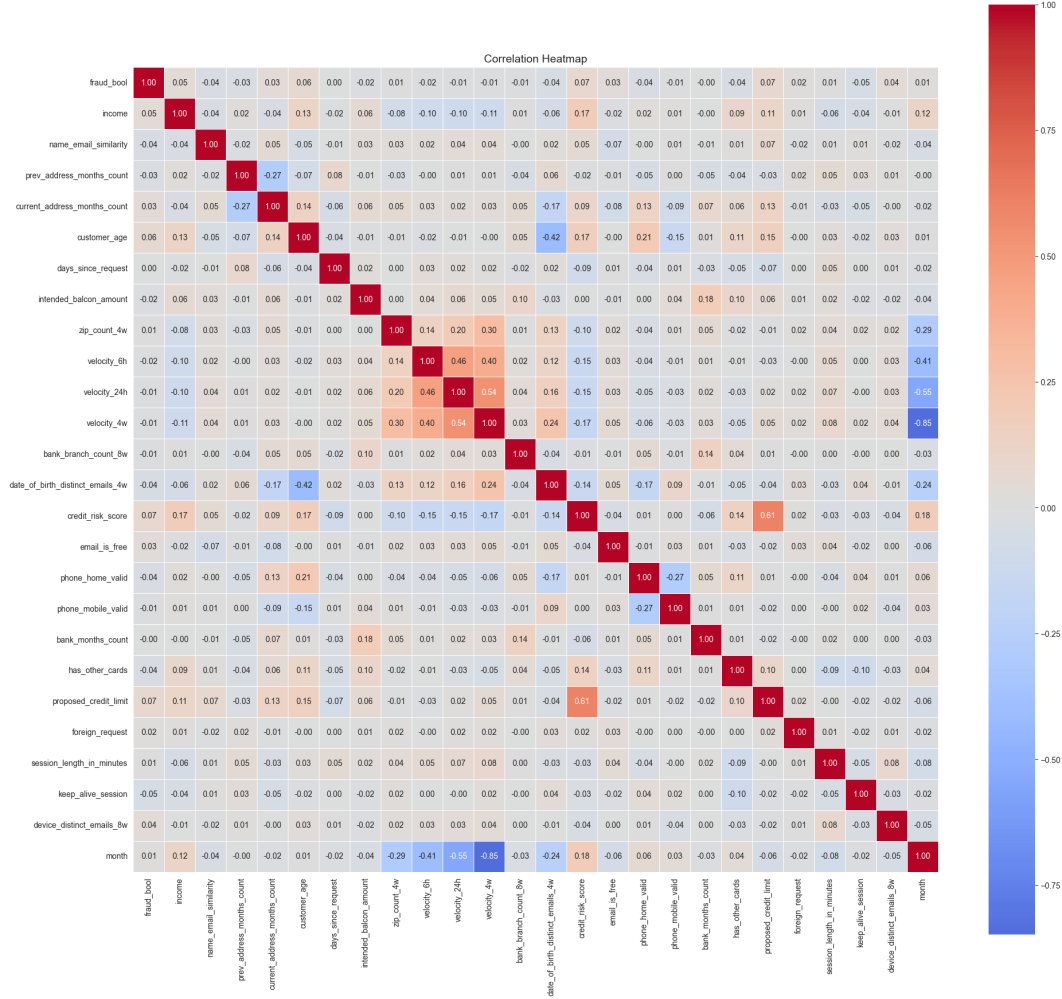


Figure 3: Correlation Heatmap

From this we see that all of the numerical variables have very weak correlation with fraud, with the highest value around 0.07, which is the feature `credit_score`. To show this more clearly, we create a plot of feature-wise correlation with the fraud indicator. As is shown in Figure 4.

### 3.3 Categorical columns

Since most correlations in the numerical variables are weak, we next examine the categorical variables. We did a categorical variable distributions strata by Target Variable graph, as shown in Figure 5.

From these we can see that the proportion of fraud is tiny compared to the overall data, due to the original data having only about 1% of fraud cases. However, even from these graphs we can see some trends. For example, in the distribution of `device_os` by target, we see that despite the fact that there are fewer users of Windows than Linux and Other in general, there are more than three times as many fraudulent cases associated with Windows than with Linux or Other. This provides a direction for further analysis and helps us understand what levels for the categorical variables can be useful split attributes in the tree based algorithms we deploy.

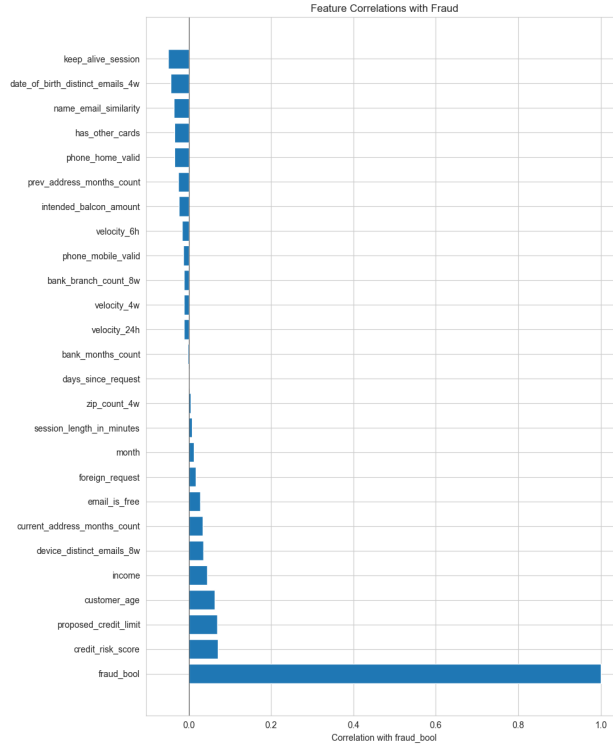


Figure 4: Feature Correlations with Fraud

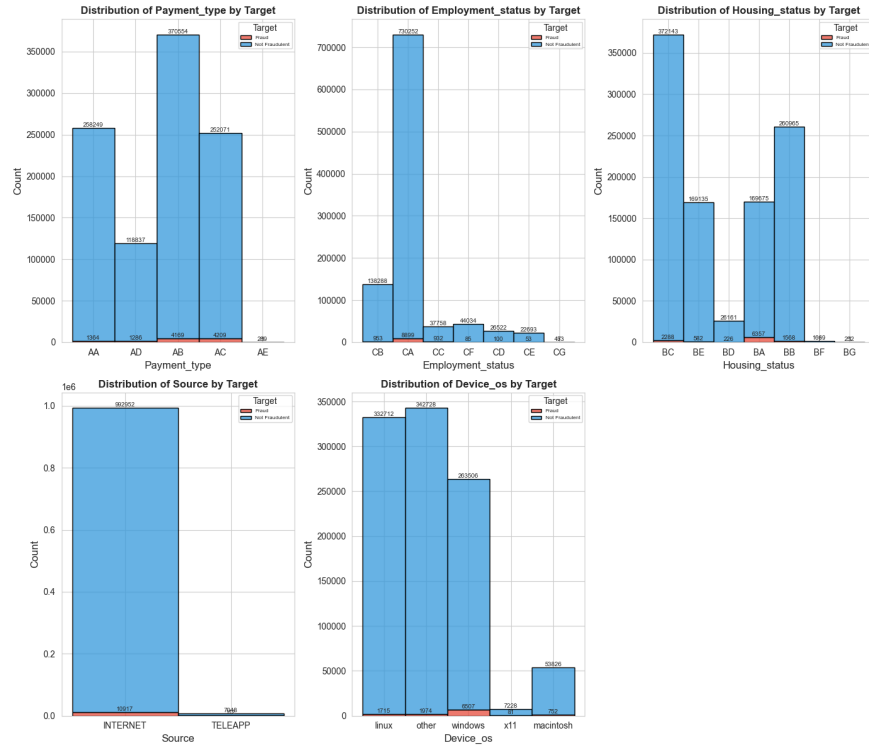


Figure 5: Categorical variable distributions strata by Target Variable

## 4 Models Development and Results

Our goal is to understand which features are most informative for building a predictive model that prioritizes classifying fraudulent cases (Our Positive class). Due to the imbalanced nature of our dataset and severity of fraudulent activity in the financial world, our primary objective is to develop a model with optimized Recall and f1-score. The reason we use Recall as the primary indicator to how well our models perform is because we want to minimize the false negatives as much as possible. Incorrectly predicting a transaction as non-fraudulent when it is fraudulent is more catastrophic than predicting a transaction as fraudulent when it actually is not.

$$\text{Formula for Recall: } Recall = \frac{TruePositive}{TruePositive+TrueNegative}$$

### 4.1 One-hot encoding and Min-Max scaling

In order to use our categorical variables in our models, we applied One-Hot encoding. This creates more variables in our design matrix; however, it allows us to use categorical variables that are quite helpful to understand whether a transaction can be fraudulent.

We also applied Min-Max Scaling for our numerical features only for our ExtraTrees model and for our Resampling technique (NearMiss) which are both discussed in the following section. We did not use the scaled data for our two primary models, Random Forest and XGBoost classifier as these tree based methods do not rely on distance based calculations to make predictions.

### 4.2 Feature Selection

Due to the large number of variables, low pairwise correlations, and certain levels for categorical features not contributing meaningfully to split observations into the fraudulent/non-fraudulent classes, we knew that we needed some sort of feature selection method in order to utilize variables with strong predictive power. The method we settled on was fitting an ExtraTrees algorithm to calculate feature importance scores. The purpose of fitting this extra tree based method was to calculate feature importance scores for all of our available variables and utilize only a subset where the importance score  $\geq$  average importance score calculated by the ExtraTrees Algorithm. The reason why we chose to use ExtraTrees for feature selection is because it is even more robust than Random Forest by fitting trees on the full dataset, with random predictors, at randomized threshold levels rather than optimized ones. The randomized split levels gets us reduced variance estimates for our feature importance scores, allowing for a more precise understanding of what attributes are actually contributing to more homogeneous splits more often of the time.

In order to fairly test the attributes splitting ability, the ExtraTrees Model was fit on a subset of the full dataset with the following shape. The subset of the data was derived with the NearMiss algorithm which under samples the dominant class observations in the dataset and keeps samples from the majority class that are most relevant to understanding the decision boundary between the positive and negative class. The algorithm uses Euclidean Distance as the primary metric to calculate similarity between the points in the two classes, which is the primary reason why Min-Max scaling was applied in our feature selection process.

*samplesperclass(BeforeNearMiss) : (0 : 988971, 1 : 11029) resampled dataset shape(AfterNearMiss) : Counter(0 : 11029, 1 : 11029)*

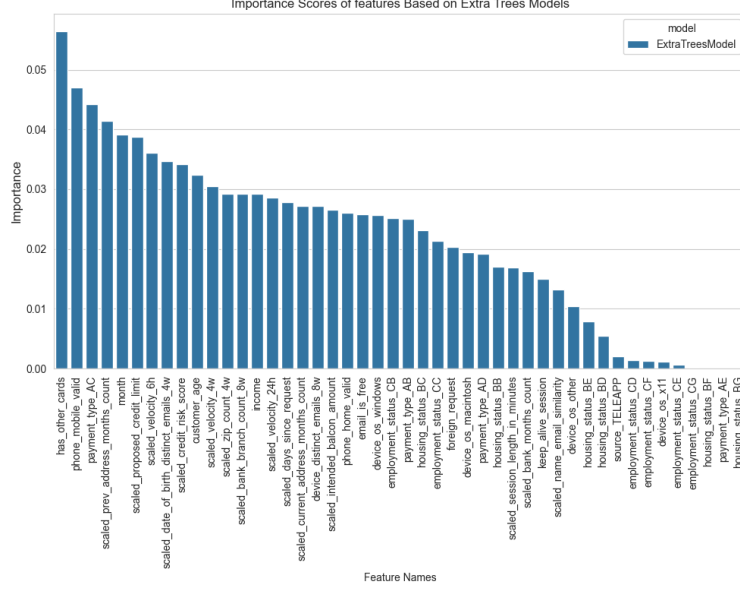


Figure 6: Importance Scores Plot calculated from ExtraTrees Model

### 4.3 Resample the Dataset for less class imbalance

Before we could fit any machine learning method into our data set, we needed to address the primary issue of large class imbalance. Once again, we deployed the NearMiss algorithm in order to under-sample our dominant class to achieve a ratio of 10 : 1 for non-fraudulent to fraudulent transactions.

samples per class: Counter(0: 988971, 1: 11029) resampled dataset shape: Counter(0: 110290, 1: 11029)

This will allow our machine learning algorithms to focus on predicting our positive class correctly rather than defaulting to a negative class prediction in order to minimize error. The NearMiss algorithm was deployed on the dataset with a reduced amount of attributes after our feature selection process. The Random Forest Model + XGBoost Model fit in the following two sections use 25 features with significant importance scores compared to 47 total predictors in our dataset after One-Hot Encoding.

### 4.4 XGBoost model

The first predictive model we fit was an XBoost Classifier model. We chose this method as its a Tree Based Learning algorithm with a variety of hyperparameters we can tune, built in L1/L2, as well simple implementation with cross-validation. We implemented 5-fold cross-validation in order to best understand which hyperparameters minimize our validation error before evaluating the XGBoost Model on our test set. Before hyperparameter tuning, we split the data in to an 80-20 training and testing dataset stratified by our target variable, ensuring we have a similar proportion of the positive class in both the training and testing splits.

After the initial split of our dataset, we created a grid search object in order to help us find the best hyperparameter selection for our model. The grid search allows us to search in a defined hyperparameter space and evaluate the current hyperparameter combination on a validation split of the training dataset (Thanks to cross-validation) to understand its performance. Our defined hyperparameter search space for the XGBoost Model is defined below:

| Table 1: XGBoost Grid Search |                    |
|------------------------------|--------------------|
| Number of Estimators         | 0, 20, 30, 50, 100 |
| Max Depth                    | 4, 8, 10, 15       |
| Learning Rate                | 0.05, 0.1, 0.2     |

With the above Grid Search object defined and 5-Fold cross validation, we are able to fit  $5*5*3*5 = 375$  total models and choose the hyperparameter combination that reduced our cross-validation loss the most. The resulting combination after exhausting the grid search is:

best parameters: 'learning rate': 0.2, 'max depth': 8, 'n estimators': 100

Our Final XGBoost model uses the above hyperparameter combination when generating predictions on our test set. Below is the confusion matrix and classification report for our models performance on the test set:

Table 2: Confusion Matrix

|            | Pred Neg | Pred Pos |
|------------|----------|----------|
| Actual Neg | 21822    | 236      |
| Actual Pos | 870      | 1336     |

Table 3: Classification Report

|           | Precision | Recall | f1-score | support |
|-----------|-----------|--------|----------|---------|
| Non-Fraud | 0.96      | 0.99   | 0.98     | 22058   |
| Fraud     | 0.85      | 0.61   | 0.71     | 2206    |

Based on the classification report, the model performed well in the Precision category for both the positive and negative class. For the positive class, the recall is 0.71, meaning we still have a large proportion of observations that are predicted as negative when they are truly in the positive class (False Positives). Even though the recall is above 0.5, future steps need to be taken in order to tune the XGBoost model to achieve a higher recall among the positive observations.

## 4.5 Random Forest

A random forest is a supervised machine learning algorithm that is constructed from decision tree algorithms and it ensembles of many decision trees that use bagging (bootstrap aggregating) and random feature selection to reduce variance and mitigate overfitting. In this model, we used the same process with XGBoost model to do the data split and evaluate model. We also use different combination of hyperparameter to train and then eventually the best-performing model achieved a cross-validation F1-score of 0.601 with hyperparameters:

best parameters: {'criterion': 'gini', 'max depth': 12, 'max features': 'sqrt', 'n estimators': 80}

Table 4: Confusion Matrix

|            | Pred Neg | Pred Pos |
|------------|----------|----------|
| Actual Neg | 21965    | 93       |
| Actual Pos | 1238     | 968      |

For the Random Forest results, the precision and recall for non-fraud cases are above 0.95, but the recall for fraud is only 0.46, which is much lower than the XGBoost models recall. In general, the performance of Random Forest is worse than that of XGBoost, as both the recall and f1-score are lower.

| Table 5: Classification Report |           |        |          |         |
|--------------------------------|-----------|--------|----------|---------|
|                                | Precision | Recall | f1-score | support |
| Non-Fraud                      | 0.95      | 1.00   | 0.97     | 22058   |
| Fraud                          | 0.91      | 0.46   | 0.59     | 2206    |

## 5 Results and Conclusion

After fitting the Random Forest and XGBoost models, we decided on the XGBoost model as our final model. This is primary due to the significant discrepancy of recall between the two methods, where XGBoost jumps to 0.71 recall from Random Forests 0.46. The criterion of selecting XGBoost as our final classifier aligns with our goal of choosing a model that reduces the number of false negatives. The only criterion that the Random Forest model performs better in is the precision for the positive class, which is outweighed by XGBoost performance for recall making it so the XGBoost classifier performs better on our imbalanced data.

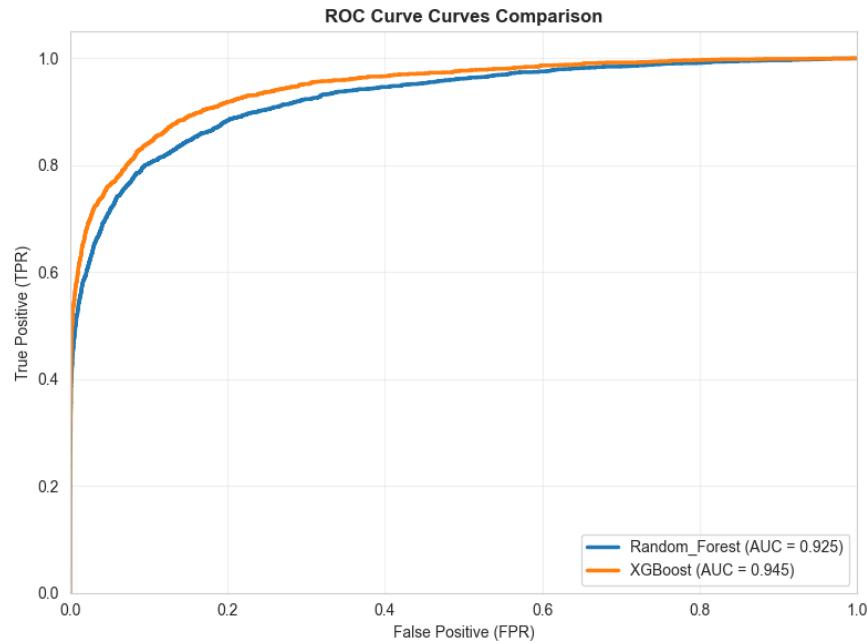


Figure 7: ROC Curves Comparison

To further emphasize our choice of XGBoost model over random forest, the ROC Curve above illustrates that the XGBoost model creates a better balance of True Positive rate vs False Negative rate compared to Random Forest's balance.

## 6 Discussion on Future Steps

In the end, we are happy with our final model results given the difficult nature of prediction on such a highly imbalanced dataset. For future implications, our final model can help the banking industry flag transactions as fraudulent.

Even though our model performed above baseline prediction levels, we know that there is still room for improvement. For example, we can expand our grid search to incorporate more combinations of hyperparameters for both grid searches objects to see whether a different area of the hyperparameter space can result in better modeling results. There can also be further work with more complex feature engineering strategies such as creating indicator variables to denote missingness for certain columns for both fraud and non-fraud cases.



Another step we can take for further modeling improvement is to utilize more strategies that can help capture more complex, non-linear trends in our data that perhaps our two current methods miss. Some examples of such models include using Gradient-Boosting models specific to imbalanced data or utilize a Deep Neural Network.

## References

- [1] OpenAI, 9 Dec 2025. <https://chatgpt.com/> - correct the grammar
- [2] Claude, 8 Dec 2025. <https://claude.ai/new> - generate for README.md to make user understand how to walk through the files.
- [3] Jeffrey G., Eugenie P. and Monica A. July 31, 2025. Pew Research Center. <https://www.pewresearch.org/internet/2025/07/31/online-scams-and-attacks-in-america-today/>
- [4] Editorial Journalist, June 30, 2025. Viva Technology. <https://vivatechnology.com/news/how-companies-use-predictive-analytics-to-detect-fraud>
- [5] Sérgio J., Pedro S., & Jose P., 2022. Bank Account Fraud Dataset Suite (NeurIPS 2022). <https://www.kaggle.com/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022/versions/1>
- [6] Kaggle <https://www.kaggle.com/code/matthewmcnulty/bank-account-fraud> - we followed the the tutorial to do part of model for the reference, but not exactly same as the model.

## Topics Covered in STT 810

- [1] Random Variables
- [2] Bootstrapping
- [3] Correlation
- [4] Feature Engineering Selection

## Github Repository link

<https://github.com/inynch/bank-fraud>